

UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Compiler Lab

Lab Report-6

Course Title : Compiler Lab

Course Code : CSE0613322

Submitted to:

Teacher's Name	Designation
Sonia Afroz	Lecturer
Tasneen Tanvir	Lecturer

Submitted by:

Student Name : Gaus Saraf Murady

Student ID : 0432410005101088

Semester : Autumn 2026

Batch : 55

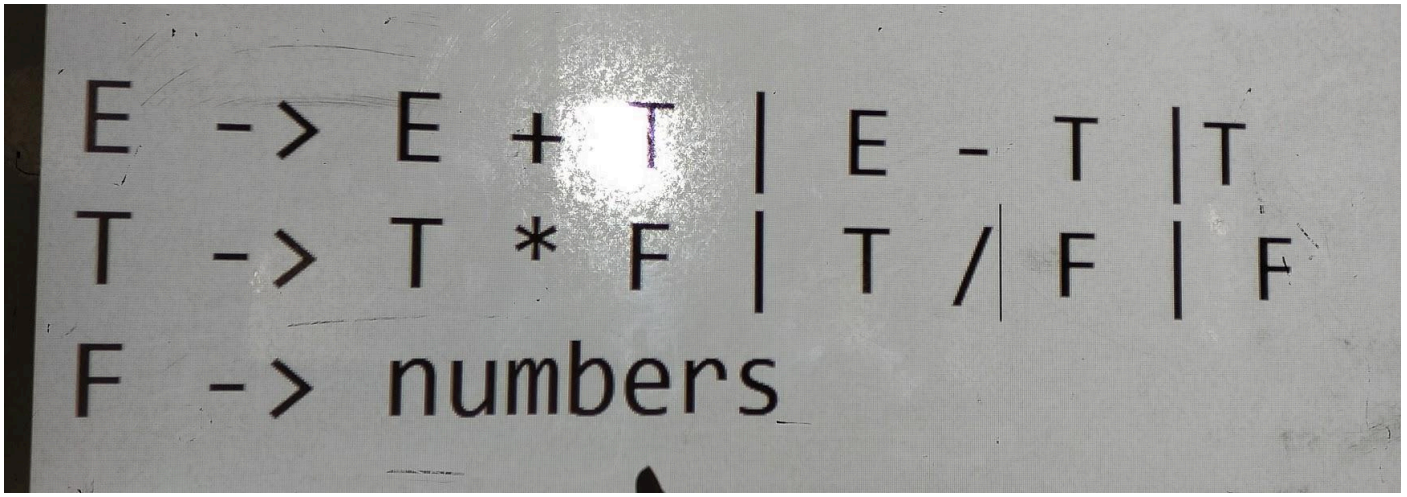
Department : CSE

Date of Submission: 10.10.26

Experiment No. : 6

Experiment Name: Math Statement Valid or invalid

Given Rule (CFG):



Code:

```
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

// Helper: check if character is a digit
bool NUM(char n) {
  return isdigit(static_cast<unsigned char>(n));
}

// Helper: check if character is an arithmetic operator
bool OP(char n) {
  return n == '+' || n == '-' || n == '*' || n == '/';
}

// Helper: check if character is a parenthesis
bool BRAC(char n) {
  return n == '(' || n == ')';
}

// Validates infix mathematical expression syntax
bool isValidExpression(const string& exp) {
  if (exp.empty()) return false;

  int n = static_cast<int>(exp.size());
  int openCount = 0;

  // Rule 1: Expression cannot start or end with a binary operator
  if (OP(exp[0]) || OP(exp[n - 1])) {
    return false;
  }

  for (int i = 0; i < n; i++) {
    char curr = exp[i];

    if (curr == '(') {
      openCount++;
      // Cannot have empty brackets `()` or operator immediately after `(`
      if (i + 1 < n && (exp[i + 1] == ')' || OP(exp[i + 1]))) {
        return false;
```

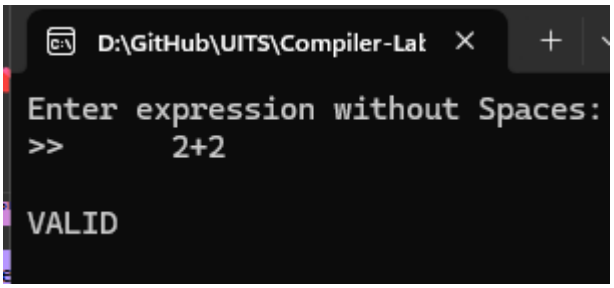
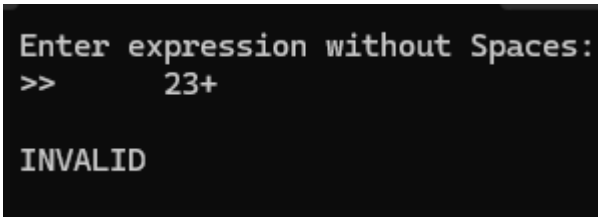
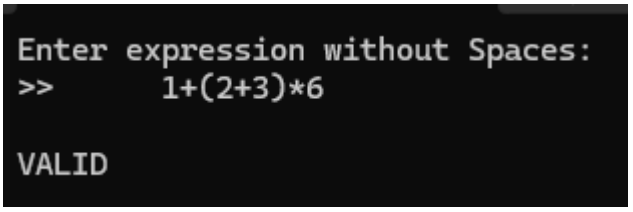
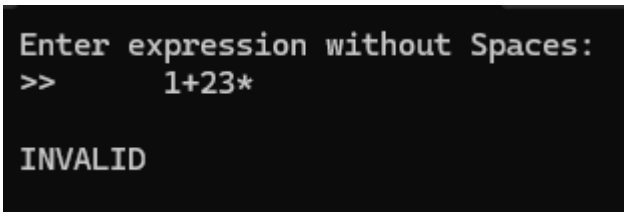
```
      }
    } else if (curr == ')') {
      openCount--;
      // Closing bracket cannot appear without a preceding open bracket
      if (openCount < 0) {
        return false;
      }
      // Closing bracket cannot be directly followed by a number or '(' without an operator
      if (i + 1 < n && (NUM(exp[i + 1]) || exp[i + 1] == '(')) {
        return false;
      }
    } else if (OP(curr)) {
      // Operator must be followed by a number or '('
      if (i + 1 < n && !(NUM(exp[i + 1]) || exp[i + 1] == '(')) {
        return false;
      }
    } else if (NUM(curr)) {
      // Number cannot be followed immediately by '(' without an explicit operator
      if (i + 1 < n && exp[i + 1] == '(') {
        return false;
      }
    } else {
      // Invalid character (spaces, unrecognized symbols)
      return false;
    }
  }

  // All open brackets must be matched
  return openCount == 0;
}

int main() {
  string exp;
  cout << "Enter expression without Spaces:\n>>\t";
  if (!getline(cin, exp)) {
    return 0;
  }
  cout << endl;

  if (isValidExpression(exp)) {
    cout << "VALID" << endl;
  } else {
    cout << "INVALID" << endl;
  }

  return 0;
}
```

Input (from Class Notes)	**Output**
2+2	
23+	
1+(2+3)*6	
1+23*	

Discussion:

Through this assignment, I learned how to identify and show output by the corresponding input string for math expressions. Here are my observations and learnings:

- I used 2 Bracket Counter variables instead of DSA stack algorithms for checking correct use of brackets, from the result of their equality. But there were some Edge cases unnoticed. Therefore the debugging AI suggested I use a single variable and set a False rule for when the number of "closing brackets" go below 0. In which case, it would mean that there was an illegal "closing bracket" without a corresponding "opening bracket" used before it.
- Using AI, I learned about `static_cast` operator in C++ and `isdigit( )` function (instead of using `if block` to check numbers 0 to 9) that helped me reduce the size of code and avoid some type errors at the same time.
- There were some other rules that I wouldn't have noticed otherwise if not I had used AI to find errors in my code:
    - "Expression cannot start or end with a binary operator"
    - "Cannot have empty brackets `()` or operator immediately after `(`"
    - "Closing bracket cannot be directly followed by a number or '(' without an operator"
    - "Operator must be followed by a number or '('"
    - "Number cannot be followed immediately by '(' without an explicit operator" *in the code I wrote myself in the lab, I only checked for single number digits, and declared invalid expression if it was a double digit (to address the input test case `23+`)*
- Alternative approach: Why Use a Stack Instead of a Scalar Counter?
    - While the current approach uses a scalar counter (openCount) suitable strictly for single-type parentheses (), standard compiler and DSA syntax checking handles heterogeneous delimiter hierarchies (()), [], {}). A Last-In, First-Out (LIFO) Stack (std::stack<char>) preserves both nesting order and delimiter types, immediately catching cross-interleaved mismatches like (] or {[)}.

Conclusion:

I was able to complete the assignment and **understand it** successfully.